

L'intelligenza artificiale nel browser

CHI PROGETTERÀ LA MIGLIOR RETE NEURALE?



Eppure...



1997

E per i gatti?





Il mondo reale è *complesso*

Apprendimento automatico (Machine Learning)

- «Modelli» che apprendono a svolgere un compito direttamente dai dati
- I dati forniscono esempi di input-output (apprendimento supervisionato)
 - $3+2 \rightarrow 5$
 -  $\rightarrow$ mela
- Più esempi ci sono, più il modello riesce a catturare informazioni importanti
- **Generalizzazione:** valutare il modello su dati “simili”, ma che non ha mai visto prima
 - $3+4 \rightarrow 7$
 -  $\rightarrow$ mela
- *Il nostro obiettivo è massimizzare la prestazione del modello su esempi non visti durante l'allenamento*
- Test dataset: ci riserviamo un po' di dati solo per questo obiettivo
 - dividiamo il nostro dataset in due: training dataset e test dataset

«Allenare» un modello

- Il modello vede gli esempi del training dataset e per ciascun esempio fornisce una risposta
- Il modello dipende da alcuni parametri (numeri) che possono essere cambiati
 - All'inizio i parametri hanno dei valori casuali (p.es. in $[-1, 1]$)
 - All'inizio la risposta sarà casuale
- Funzione di costo (loss): misura quanto il modello è distante dalla risposta corretta
- **Ottimizzazione:** cambiare i parametri in modo tale che il modello dia più risposte corrette di prima
 - L'ottimizzazione sposta i parametri in una direzione che riduce la funzione di costo
- Tasso di apprendimento (**learning rate**)
 - $[0, 1]$
 - Più alto è, più veloce è l'apprendimento, ma anche più instabile
- **Vogliamo che la funzione di costo si riduca mano a mano che vediamo più esempi**
 - cioè che l'**accuratezza** della predizione aumenti

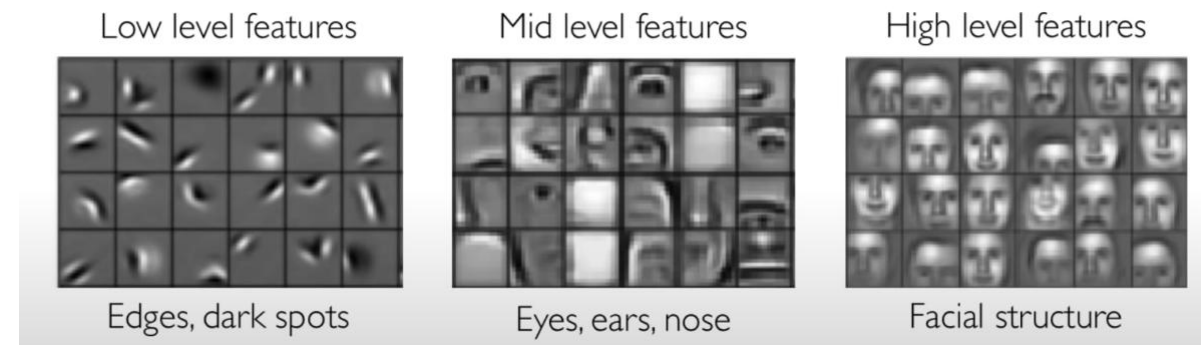
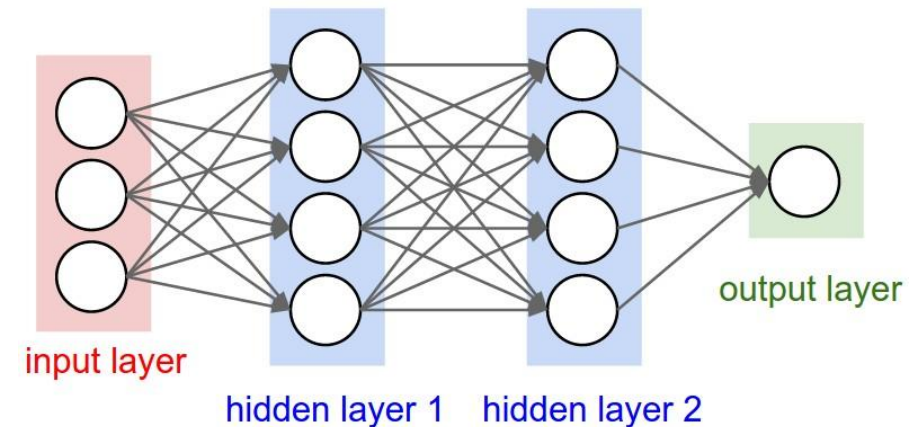
Algoritmo di allenamento

- **Inizio di un'epoca di allenamento**
- Per ogni esempio del training dataset
 - Calcoliamo la risposta del modello
 - Calcoliamo la funzione di costo (l'errore)
 - Aggiorniamo i parametri del modello per ridurre l'errore sugli esempi
- **Fine di un'epoca di allenamento**
- Ripetiamo per tante epoche, fino a quando la funzione di costo non smette di diminuire

Dato che il test dataset va usato solo alla fine, solitamente durante l'allenamento misuriamo la prestazione su un *validation dataset*, ottenuto dividendo ulteriormente il training dataset in 2

Deep Learning – Machine Learning con reti neurali artificiali

- Un modello composto da tanti *strati* (*layer*)
 - Ciascuno strato fa le stesse operazioni, ma applicate sul risultato dello strato precedente → profondità
 - Ciascuno strato è composto da tante unità → più unità, più potente è lo strato
- Più strati, più potente è la rete
- Pochi strati/unità → underfitting
 - La rete non è abbastanza potente per risolvere il compito (bassa prestazione in training)
- Troppi strati/unità → overfitting
 - La rete memorizza i dati di training senza imparare nulla
 - alta prestazione in training, bassa prestazione in test
- Come scegliere i numeri giusti? → provare!



Glossario finale

Numero di strati (layer): determina la profondità della rete, più sono più la rete è potente, più è difficile da allenare

Numero di unità: determina la potenza di un layer, come sopra

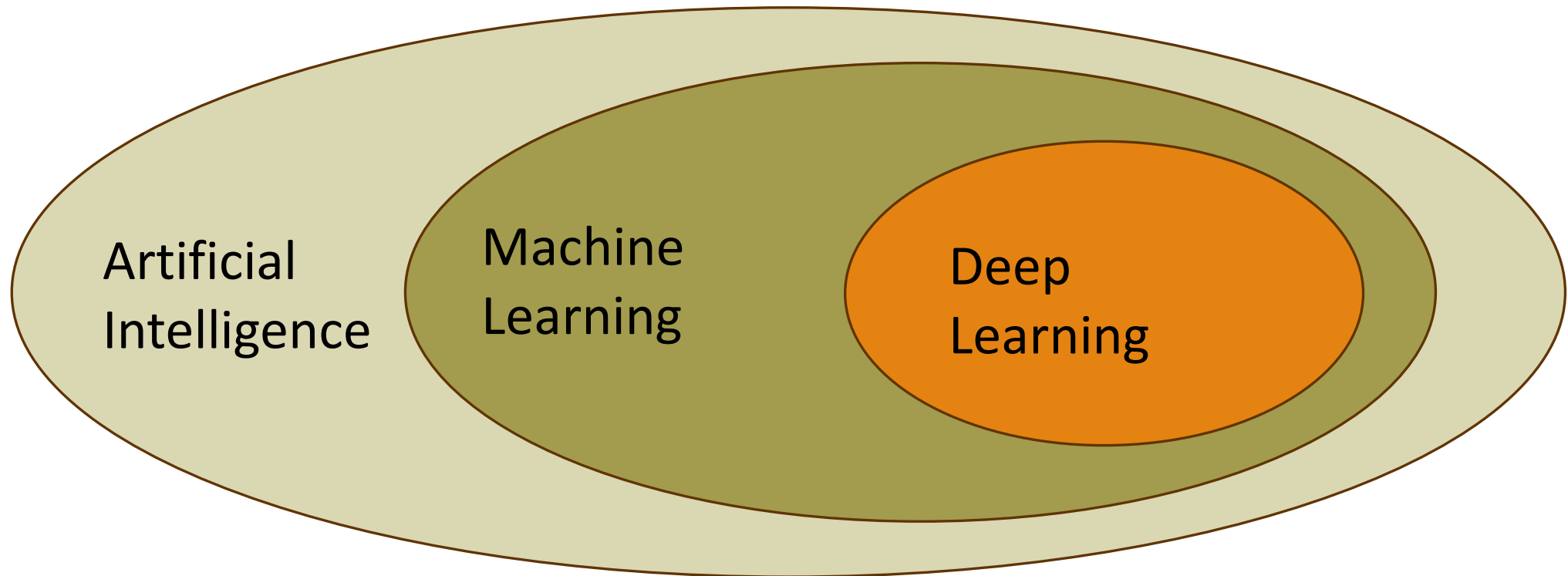
Learning rate: determina la velocità di apprendimento. Più è alto, più l'allenamento è veloce, ma più è instabile

Numero di epoche: quanti «giri» fare sul training set durante l'allenamento

Provate a cambiare questi valori nell'esempio e vedere come cambiano le prestazioni!

Training/validation/test dataset: dati su cui allenare il modello (training), valutare la prestazione durante l'allenamento (validation) e valutare la prestazione finale del modello (test).

Intelligenze artificiali





Il nostro obiettivo

Ogni cifra è un'immagine separata

Utilizzare deep learning e reti neurali artificiali per riconoscere (classificare) la cifra contenuta in ciascuna immagine

L'ultimo strato della rete avrà 10 unità, una per ciascuna possibile cifra.

L'unità con il valore maggiore sarà selezionata come risposta.



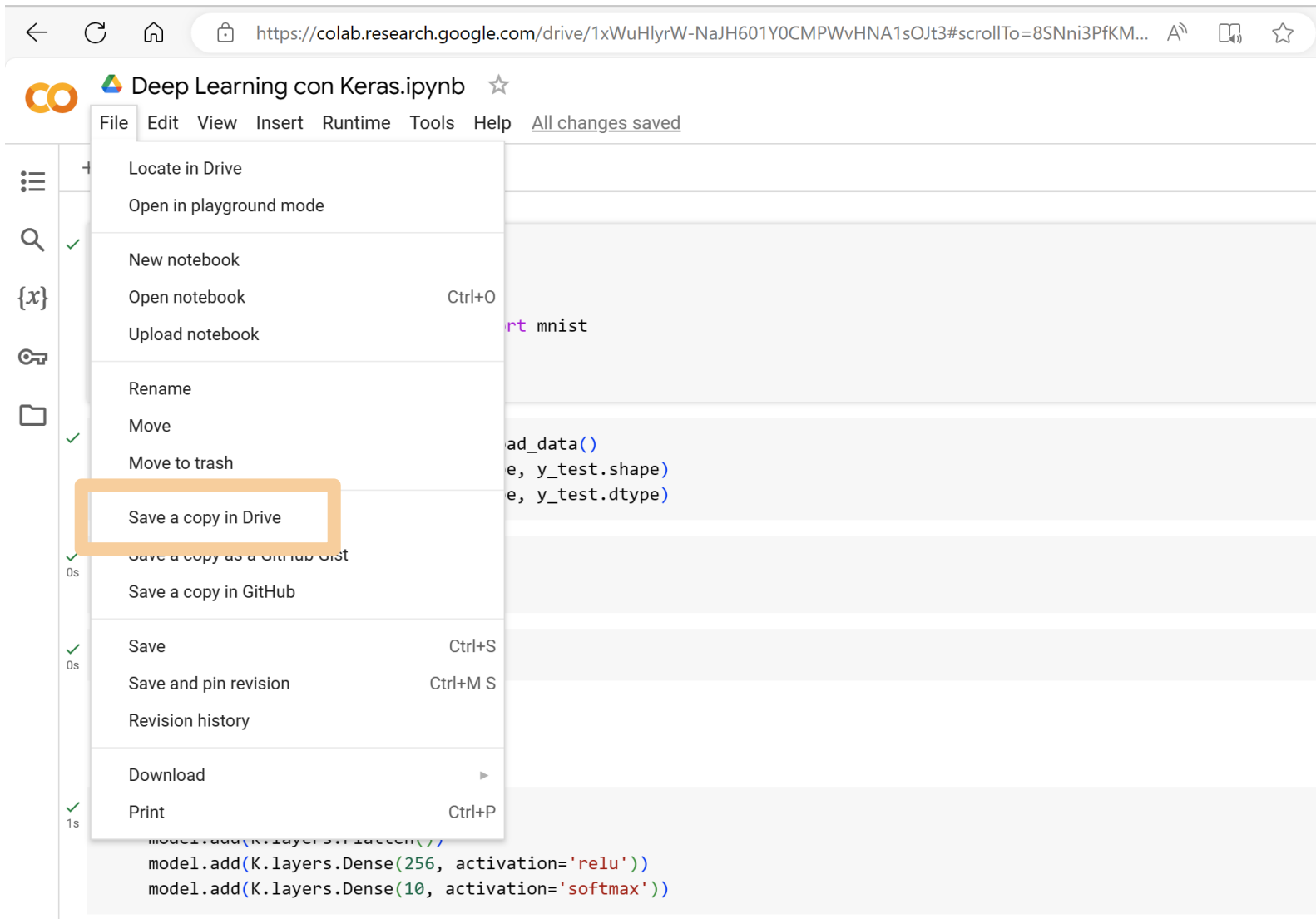
Python per il Deep Learning

- Il linguaggio più usato per il Deep Learning
 - Librerie che consentono facilmente di fare esperimenti
 - Integrato in tutte le piattaforme
 - Semplice da utilizzare
-
- Keras è una delle librerie più popolari per il Deep Learning



Google Colab

- Strumento sviluppato e gestito da Google
 - Richiede di avere un **account Google (gmail)** per avere accesso
 - Fornisce un «computer virtuale» con cui eseguire il codice
 - Il vostro computer non fa troppo lavoro
- Le principali librerie software sono già pronte all'uso → non c'è bisogno di installazione
- Consente di inserire del testo, oltre al codice
- Interattivo, può eseguire «pezzi» di codice
- Ogni «notebook» colab può essere condiviso tramite link
 - Chi riceve la condivisione può copiare il notebook e modificarlo a piacimento



Appena entrati su Colab

Create la vostra copia del notebook sul vostro Google Drive

La copia si aprirà automaticamente nel browser

Potete recuperare il notebook anche andando sul vostro Google Drive, lo trovate nella cartella "Colab Notebooks" che è stata creata

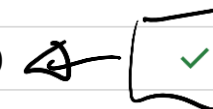
Come eseguire codice

- Il contenuto di un notebook colab è diviso in «celle»
- Ogni cella contiene o del testo o del codice
- Le celle con il codice possono essere eseguite cliccando al loro interno e premendo Ctrl+Invio, oppure cliccando sull'apposito bottone dall'interfaccia
- La prima volta che eseguite il codice il sistema impiegherà più tempo perché deve assegnarvi un computer virtuale sul quale il codice sarà eseguito
- L'ordine con cui eseguite le celle conta!



+ Code + Text

COMPUTER VIRTUALE PRONTO



T4 RAM Disk

▼ Laboratorio di Deep Learning con Keras

Alleneremo una rete neurale artificiale a riconoscere le cifre all'interno di alcune immagini.

CELLA DI TESTO

CELLA DI CODICE

CELLA DI CODICE

PULSANTE PER ESEGUIRE IL CODICE DELLA CELLA



Iniziamo!

<https://bit.ly/incontrainformaticadl>